

Functional Programming

CS 22B, Spring 2026

Module 03: Function

Presented by Jessica Huynh-Westfall, Ph.D.

AGENDA



Generator

A **generator** function is a special kind of function that you can use to define your own iterators.

When you define a generator function, you return each iteration value using the `yield` keyword

- `yield` pauses the execution and returns a value to the caller

Benefit of generatory

- Memory efficient
- Lazy evaluation - code only runs when a value is requested of the geneatory object
- Good for large data

Overview of Generator

05

Generators produce values one at a time instead of storing everything in memory.

Method 1: List expression

```
nums = [x*x for x in range(1000000)]
```

Uses lots of memory

Method 2: Generator expression

```
nums = (x*x for x in range(1000000))
```

Produces values one at a time

Generator object example

06

Use generator expression when we want to use an iterator in for loop. Generator object produces value one at a time when requested. Value is not stored in memory.

```
xs = [1, 2, 3, 4, 5]
xs_squared_gen = (xs_item * xs_item for xs_item in xs)
```

Calling a generator

```
print(xs_squared_gen)
print(next(xs_squared_gen))
```

Class exercise

07

CL 8.1

```
def count_up_to(n):  
    i = 1  
    while i <= n:  
        yield i  
        i += 1  
    print(i, "hello")
```

What gets printed? What is the difference between the output?

```
genobj1=testgen()  
for x in genobj1:  
    print(x)
```

```
genobj2 = count_up_to(5)  
print("next(genobj2): ", next(genobj2))
```

Class exercise

08

CL 8.2

Rewrite this procedurally written code using a generator expression with no intermediate lists.

```
total = 0
for i in range(1, 21):
    if i % 2 == 0:
        total += i**2
    else:
        total += i**3

print(total)
```